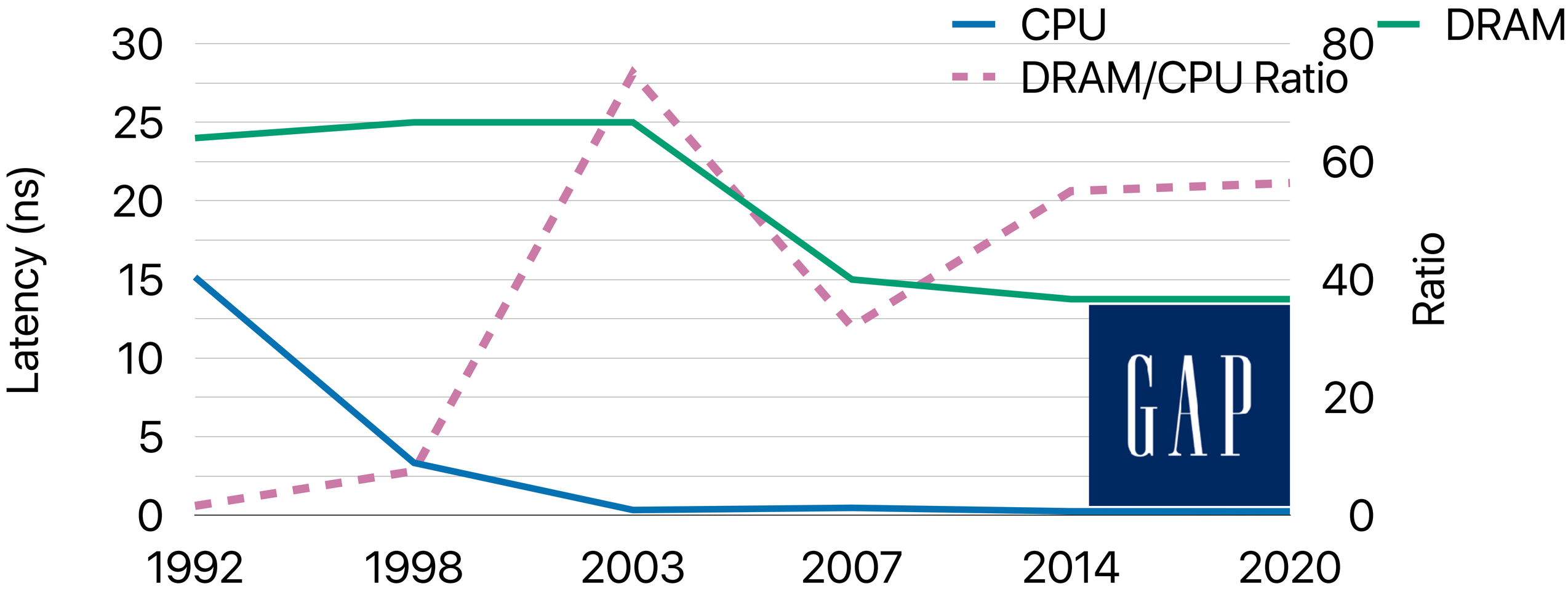
Memory hierarchy (3): Cache misses and their remedies

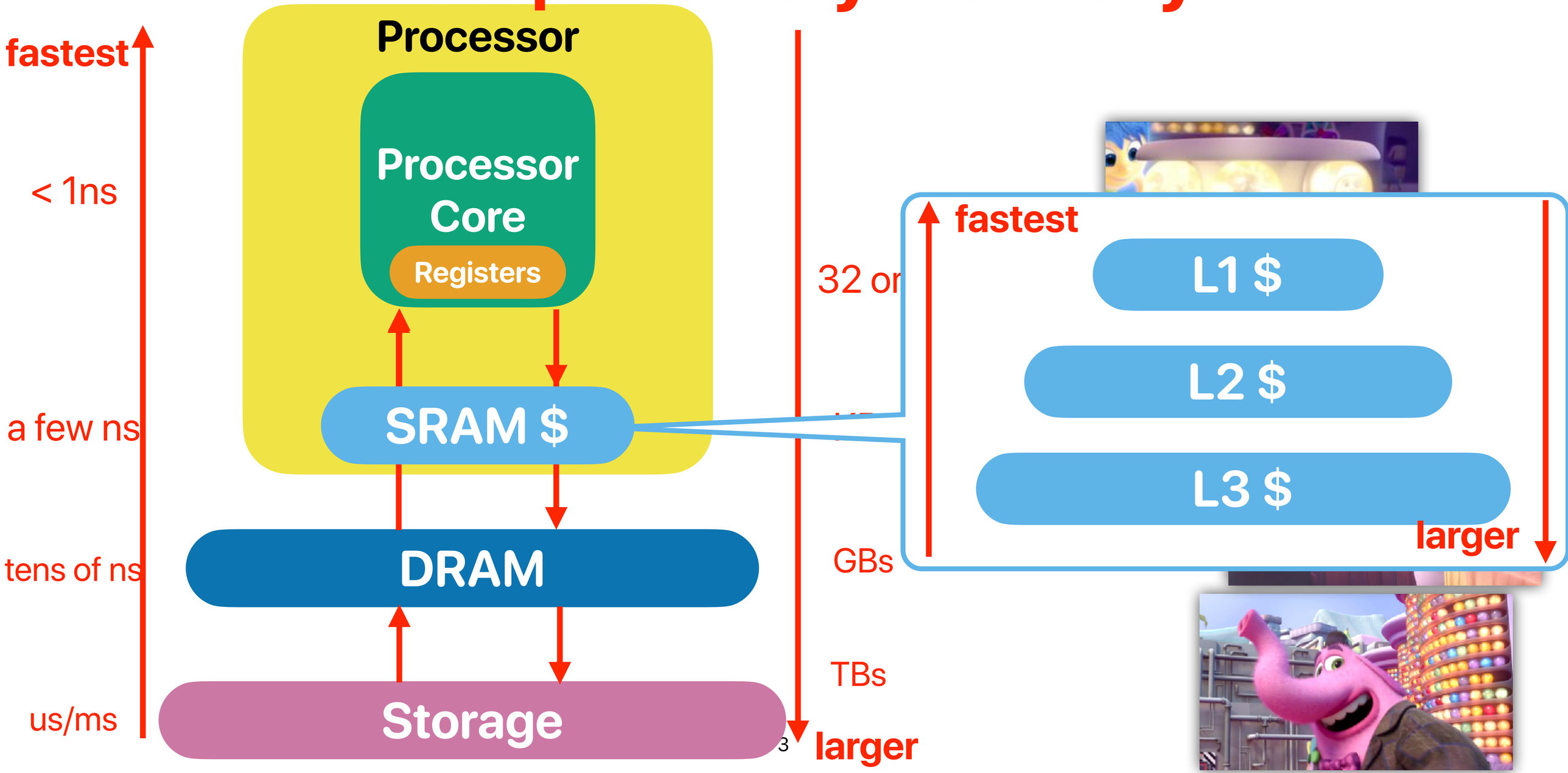
Hung-Wei Tseng

The “latency” gap between CPU and DRAM

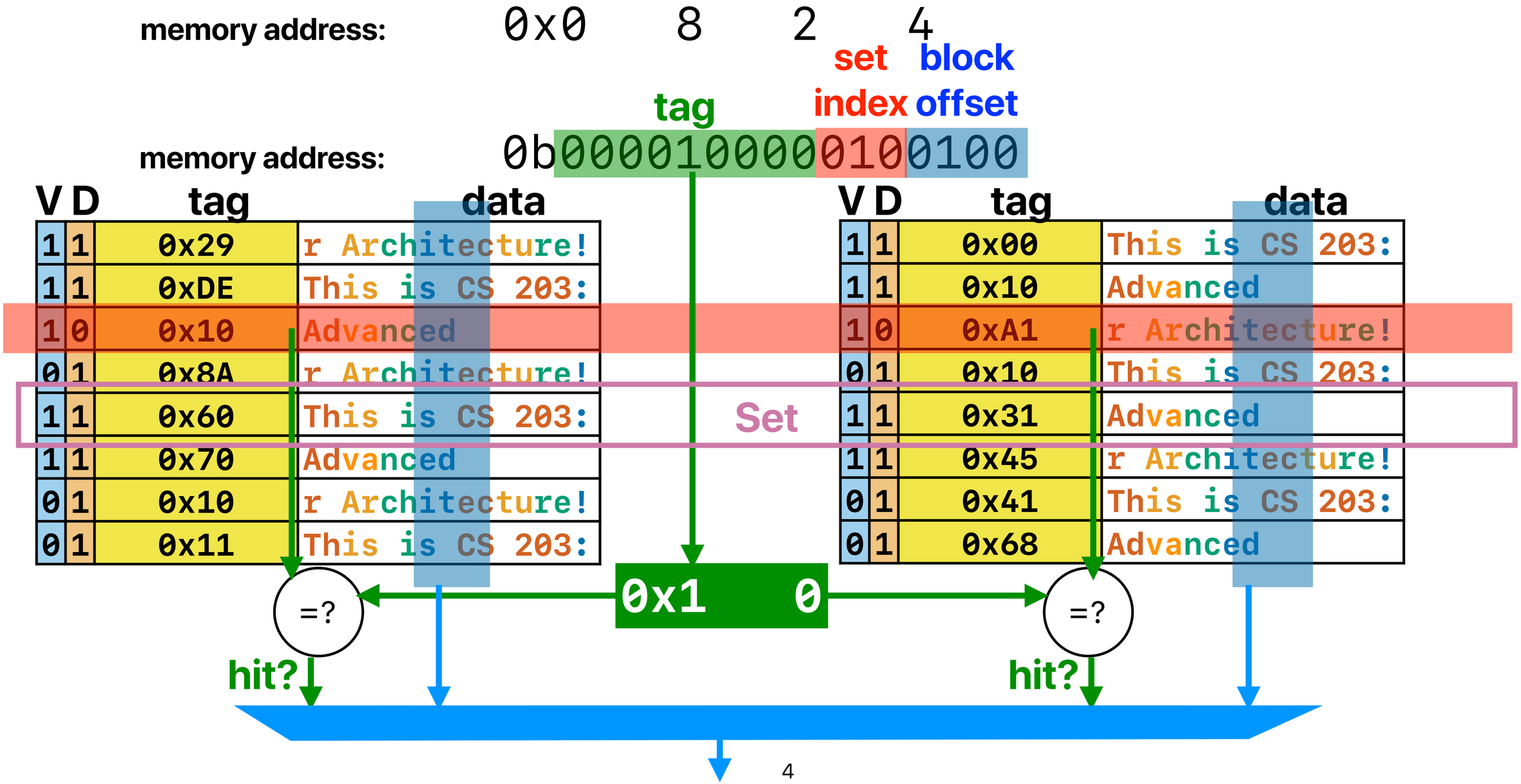


CPU	i486	Pentium II	Pentium 4	Core 2	Core i7-4790K	Core i5-10600K
DRAM	SDRAM	DDR	DDR2	DDR3	DDR4	DDR5

Recap: Memory Hierarchy



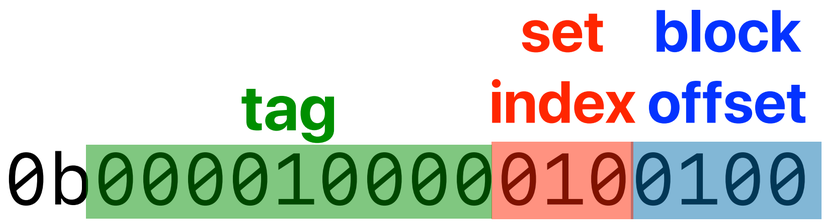
Way-associative cache



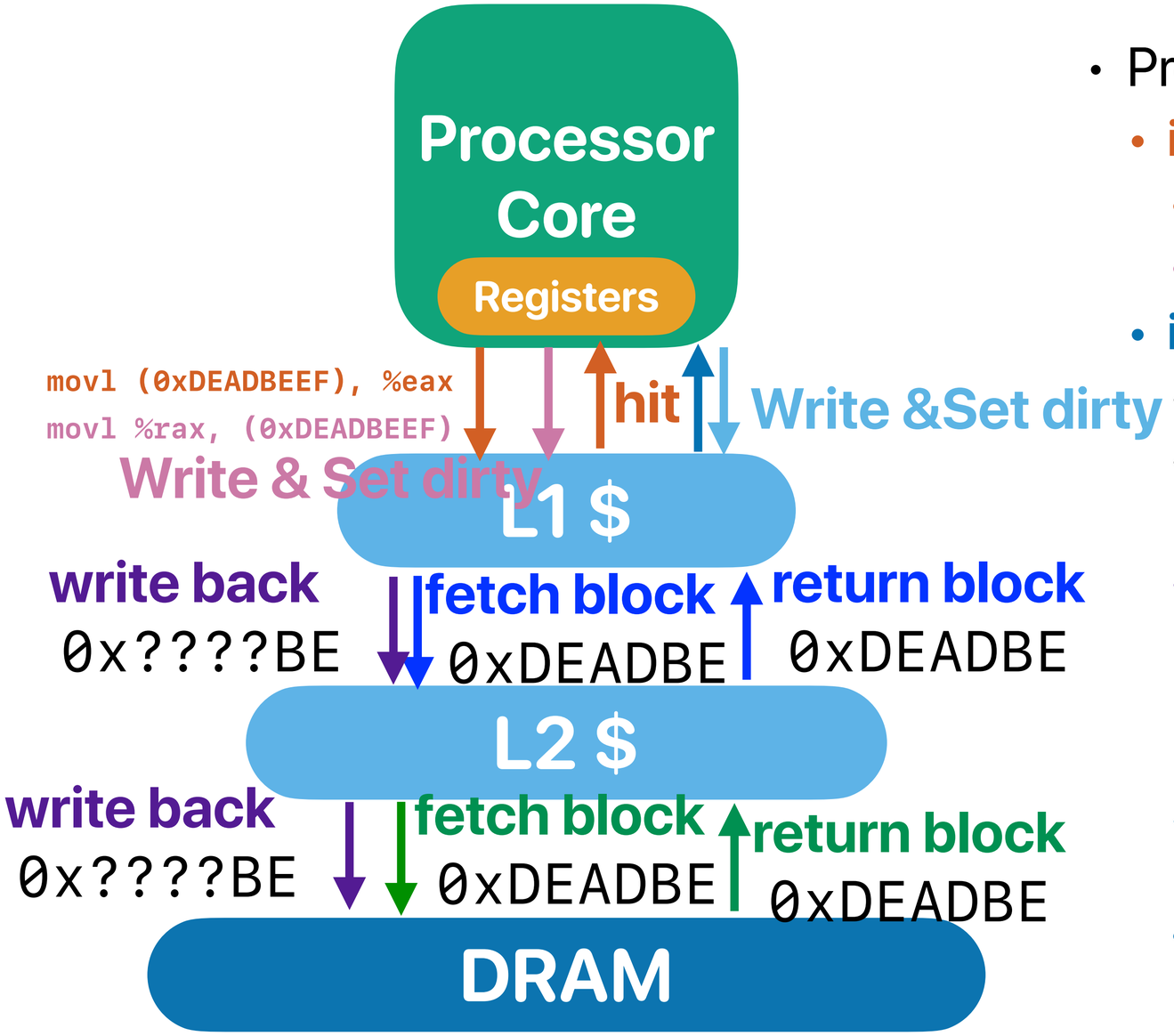
Recap: C = ABS

- **C**: **C**apacity in data arrays
- **A**: Way-**A**ssociativity — how many blocks within a set
  - N-way: N blocks in a set, A = N
  - 1 for direct-mapped cache
- **B**: **B**lock Size (Linesize)
  - How many bytes in a block
- **S**: Number of **S**ets:
  - A set contains blocks sharing the same index
  - 1 for fully associate cache
- number of bits in **b**lock offset — lg(**B**)
- number of bits in **s**et index: lg(**S**)
- tag bits: address_length - lg(S) - lg(B)
  - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- (address / block_size) % S = set index

memory address:



Processor/cache interaction



- Processor sends memory access request to L1-$
  - **if hit & it's a read**
    - **Read: return data**
    - **Write: Update "ONLY" in L1 and set DIRTY**
  - **if miss**
    - If there an empty block — place the data there
    - If NOT (most frequent case) — select a **victim block**
      - Least Recently Used (LRU) policy
    - If the victim block is "dirty" & "valid"
      - **Write back** the block to lower-level memory hierarchy
      - If write-back or fetching causes any miss, repeat the same process
    - Fetch the requesting block from lower-level memory hierarchy and place in the cache
    - **Present the write "ONLY" in L1 and set DIRTY**

NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS

64KB = 4 * 64 * S

S = 256

offset = lg(64) = 6 bits

index = lg(256) = 8 bits

tag = the rest bits

	Address (Hex)	Address in binary (tag | index | offset)	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b000100 00000000 000000	0x4	0x0	Miss	
b[0]	0x20000	0b001000 00000000 000000	0x8	0x0	Miss	
c[0]	0x30000	0b001100 00000000 000000	0xC	0x0	Miss	
d[0]	0x40000	0b010000 00000000 000000	0x10	0x0	Miss	
e[0]	0x50000	0b010100 00000000 000000	0x14	0x0	Miss	a[0–7]
a[1]	0x10008	0b000100 00000000 001000	0x4	0x0	Miss	b[0–7]
b[1]	0x20008	0b001000 00000000 001000	0x8	0x0	Miss	c[0–7]
c[1]	0x30008	0b001100 00000000 001000	0xC	0x0	Miss	d[0–7]
d[1]	0x40008	0b010000 00000000 001000	0x10	0x0	Miss	e[0–7]
e[1]	0x50008	0b010100 00000000 001000	0x14	0x0	Miss	a[0–7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
  - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

A. 12.5%
B. 56.25%
C. 66.67%
D. 68.75%
E. 100%

Taxonomy/reasons of cache misses

3Cs of misses

- Compulsory miss
  - Cold start miss. First-time access to a block
- Capacity miss
  - The working set size of an application is bigger than cache size
  - The working set size is the size of data blocks visited between the last observation point and the current point — **not** the volume of your data structure
- Conflict miss
  - Required data replaced by block(s) mapping to the same set
  - Similar collision in hash
  - The working set size is smaller than the cache capacity

https://www.pollev.com/hungweitseng close in 1:30

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
  - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

A. Compulsory misses
B. Conflict misses
C. Capacity misses
D. Capacity and conflict misses
E. They are all equally critical



NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
64KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

tag index offset

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x8	0x0	Compulsory Miss	
b[0]	0x20000	0b0010000000000000000000	0x10	0x0	Compulsory Miss	
c[0]	0x30000	0b0011000000000000000000	0x18	0x0	Compulsory Miss	
d[0]	0x40000	0b0100000000000000000000	0x20	0x0	Compulsory Miss	
e[0]	0x50000	0b0101000000000000000000	0x28	0x0	Compulsory Miss	a[0–7]
a[1]	0x10008	0b0001000000000000001000	0x8	0x0	**Conflict Miss**	b[0–7]
b[1]	0x20008	0b0010000000000000001000	0x10	0x0	**Conflict Miss**	c[0–7]
c[1]	0x30008	0b0011000000000000001000	0x18	0x0	**Conflict Miss**	d[0–7]
d[1]	0x40008	0b0100000000000000001000	0x20	0x0	**Conflict Miss**	e[0–7]
e[1]	0x50008	0b0101000000000000001000	0x28	0x0	**Conflict Miss**	a[0–7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮

NVIDIA Tegra Orin (cont.)

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
64KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

tag index offset

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b0001000000000111000	0x8	0x0	**Conflict Miss**	
b[7]	0x20038	0b0010000000000111000	0x10	0x0	**Conflict Miss**	
c[7]	0x30038	0b0011000000000111000	0x18	0x0	**Conflict Miss**	
d[7]	0x40038	0b0100000000000111000	0x20	0x0	**Conflict Miss**	
e[7]	0x50038	0b0101000000000111000	0x28	0x0	**Conflict Miss**	a[0–7]
a[8]	0x10040	0b0001000000001000000	0x8	0x1	Compulsory Miss	
b[8]	0x20040	0b0010000000001000000	0x10	0x1	Compulsory Miss	
c[8]	0x30040	0b0011000000001000000	0x18	0x1	Compulsory Miss	
d[8]	0x40040	0b0100000000001000000	0x20	0x1	Compulsory Miss	
e[8]	0x50040	0b0101000000001000000	0x28	0x1	Compulsory Miss	a[8–15]
⋮	⋮	⋮	⋮	⋮	⋮	⋮

100% miss rate!

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
  - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

A. Compulsory misses
B. Conflict misses
C. Capacity misses
D. Capacity and conflict misses
E. They are all equally critical

https://www.pollev.com/hungweitseng close in 1:30

intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-core
  - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

A. Compulsory misses
B. Conflict misses
C. Capacity misses
D. Capacity and conflict misses
E. They are all equally critical



intel Core i7 14700K

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

tag index offset

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x10	0x0	Compulsory Miss	
b[0]	0x20000	0b0010000000000000000000	0x20	0x0	Compulsory Miss	
c[0]	0x30000	0b0011000000000000000000	0x30	0x0	Compulsory Miss	
d[0]	0x40000	0b0100000000000000000000	0x40	0x0	Compulsory Miss	
e[0]	0x50000	0b0101000000000000000000	0x50	0x0	Compulsory Miss	
a[1]	0x10008	0b0001000000000000001000	0x10	0x0	Hit	
b[1]	0x20008	0b0010000000000000001000	0x20	0x0	Hit	
c[1]	0x30008	0b0011000000000000001000	0x30	0x0	Hit	
d[1]	0x40008	0b0100000000000000001000	0x40	0x0	Hit	
e[1]	0x50008	0b0101000000000000001000	0x50	0x0	Hit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

intel Core i7 14700K (cont.)

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

tag index offset

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b0001000000000111000	0x10	0x0	Hit	
b[7]	0x20038	0b0010000000000111000	0x20	0x0	Hit	
c[7]	0x30038	0b0011000000000111000	0x30	0x0	Hit	
d[7]	0x40038	0b0100000000000111000	0x40	0x0	Hit	
e[7]	0x50038	0b0101000000000111000	0x50	0x0	Hit	
a[8]	0x10040	0b0001000000001000000	0x10	0x1	Compulsory Miss	
b[8]	0x20040	0b0010000000001000000	0x20	0x1	Compulsory Miss	
c[8]	0x30040	0b0011000000001000000	0x30	0x1	Compulsory Miss	
d[8]	0x40040	0b0100000000001000000	0x40	0x1	Compulsory Miss	
e[8]	0x50040	0b0101000000001000000	0x50	0x1	Compulsory Miss	
a[9]	0x10048	0b0001000000001001000	0x10	0x1	Hit	
b[9]	0x20048	0b0010000000001001000	0x20	0x1	Hit	
c[9]	0x30048	0b0011000000001001000	0x30	0x1	Hit	
d[9]	0x40048	0b0100000000001001000	0x40	0x1	Hit	

intel Core i7 14700K

- D-L1 Cache configuration of intel Core i7 14700K's P-core
  - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

A. Compulsory misses
B. Conflict misses
C. Capacity misses
D. Capacity and conflict misses
E. They are all equally critical

Take-aways: cache misses and their remedies

- Our code behaves differently on different cache configurations
- Cache misses
  - Compulsory misses — the miss due to first time access of a block
  - Capacity misses — the miss due to the working set size surpasses the **capacity**
  - Conflict misses — the miss due to insufficient blocks of the target set **(associativity)**

A, B, C, S and cache misses

https://www.pollev.com/hungweitseng close in 1:30

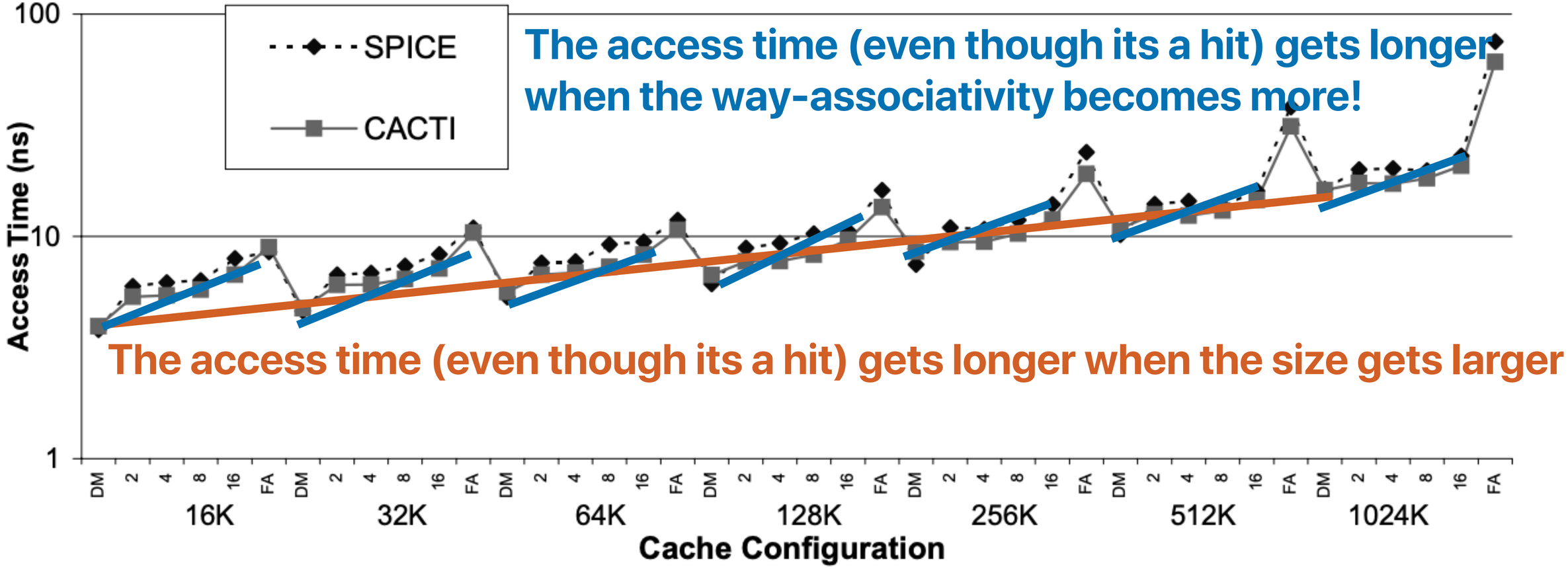
3Cs and A, B, C

- Regarding 3Cs: compulsory, conflict and capacity misses and
  A, B, C: associativity, block size, capacity
  How many of the following are correct?
  - ① Without changing B & C, increasing A can reduce conflict misses but make each cache hit slower
  - ② Without changing A & C, increasing B can reduce compulsory misses but potentially lead to more conflict misses
  - ③ Without changing A & C, increasing B will make each cache miss slower
  - ④ Without changing A & B, increasing C can reduce capacity misses but make each cache hit slower

  A. 0
  B. 1
  C. 2
  D. 3
  E. 4

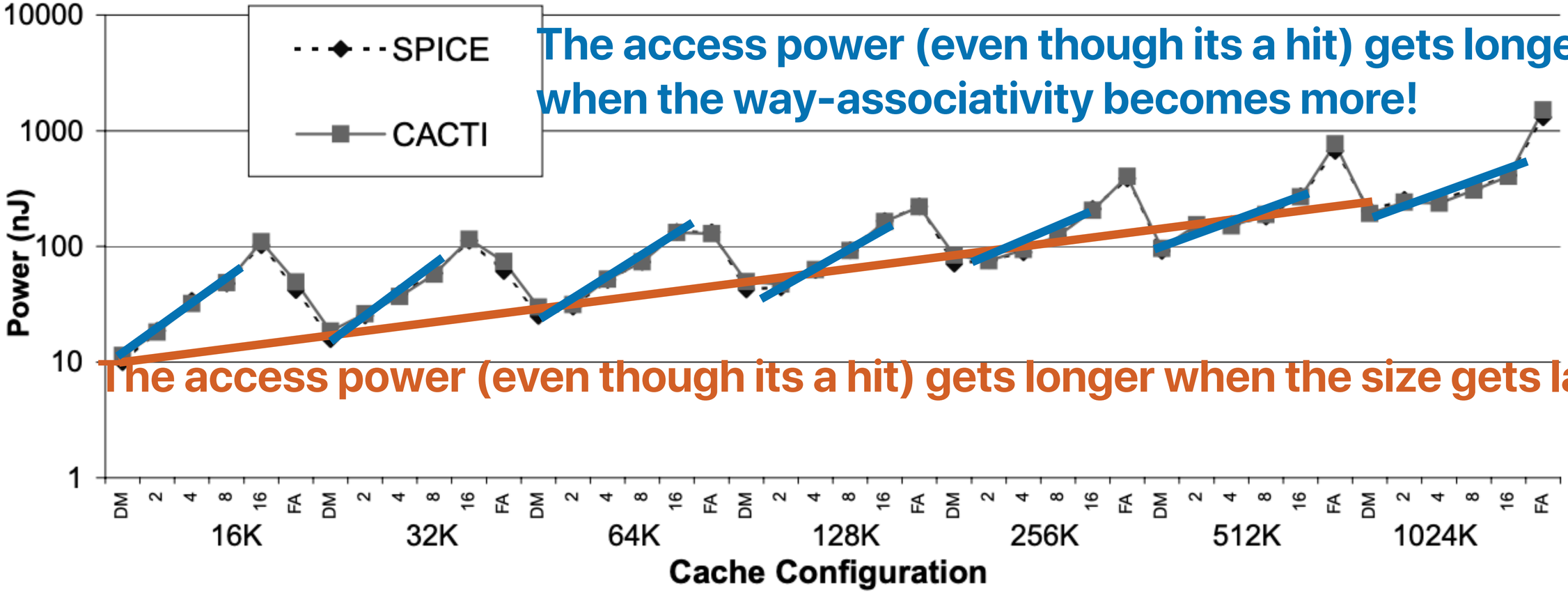


Cache configurations and accessing time



Glen Reinman and Norman P. Jouppi. CACTI 2.0: An Integrated Cache Timing and Power Model. WRL Research Report 2000/7
https://bitsavers.trailing-edge.com/pdf/dec/tech_reports/WRL-2000-7.pdf

Cache configurations and accessing power



Glen Reinman and Norman P. Jouppi. CACTI 2.0: An Integrated Cache Timing and Power Model. WRL Research Report 2000/7
https://bitsavers.trailing-edge.com/pdf/dec/tech_reports/WRL-2000-7.pdf

3Cs and A, B, C

- Regarding 3Cs: compulsory, conflict and capacity misses and A, B, C: associativity, block size, capacity
  How many of the following are correct?
  ① Without changing B & C, increasing A can reduce conflict misses but make each cache hit slower
  ② Without changing A & C, increasing B can reduce compulsory misses but potentially lead to more conflict misses
  ③ Without changing A & C, increasing B will make each cache miss slower
  ④ Without changing A & B, increasing C can reduce capacity misses but make each cache hit slower

  A. 0
  B. 1
  C. 2
  D. 3
  E. 4

You need to compare more tags

You reduce the number of sets now

You bring more into the cache when a miss occurs

Increases hit time because your data array is larger (longer time to fully charge your bit-lines)

You need to fetch more data for each miss

Take-aways: cache misses and their remedies

- Our code behaves differently on different cache configurations
- Cache misses
  - Compulsory misses — the miss due to first time access of a block
  - Conflict misses — the miss due to insufficient blocks of the target set **(associativity)**
  - Capacity misses — the miss due to the working set size surpasses the **capacity**
- There is no optimal cache configurations — trade-offs are everywhere
  - Increasing C — (+): capacity misses; (-): cost, access time, power
  - Increasing A — (+): conflict misses; (-): access time, power
  - Increasing B — (+): compulsory misses; (-): miss penalty

How can we improve cache performance without changing ABCs?

Recap: NVIDIA Tegra X1

- D-L1 Cache configuration of NVIDIA Tegra X1
  - Size 32KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[8192], b[8192], c[8192], d[8192], e[8192];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

A. 12.5%
B. 56.25%
C. 66.67%
D. 68.75%
E. 100%

Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers

Norman P. Jouppi



https://www.pollev.com/hungweitseng close in 1:30

Which of the following schemes can help NVIDIA Orin?

- How many of the following schemes mentioned in "improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers" would help NVIDIA's Tegra for the code in the previous slide?
  - ① Missing cache
  - ② Victim cache
  - ③ Prefetch
  - ④ Stream buffer

A. 0
B. 1
C. 2
D. 3
E. 4

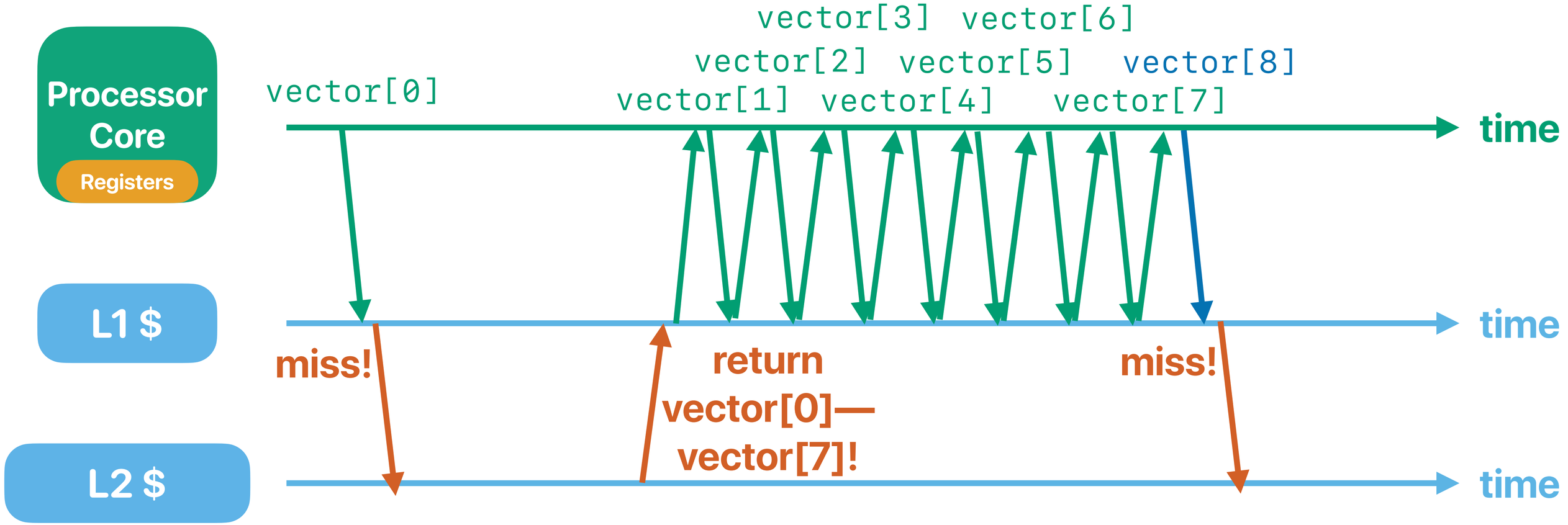
```
double a[8192], b[8192], c[8192], d[8192], e[8192];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 8192; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```



Prefetching

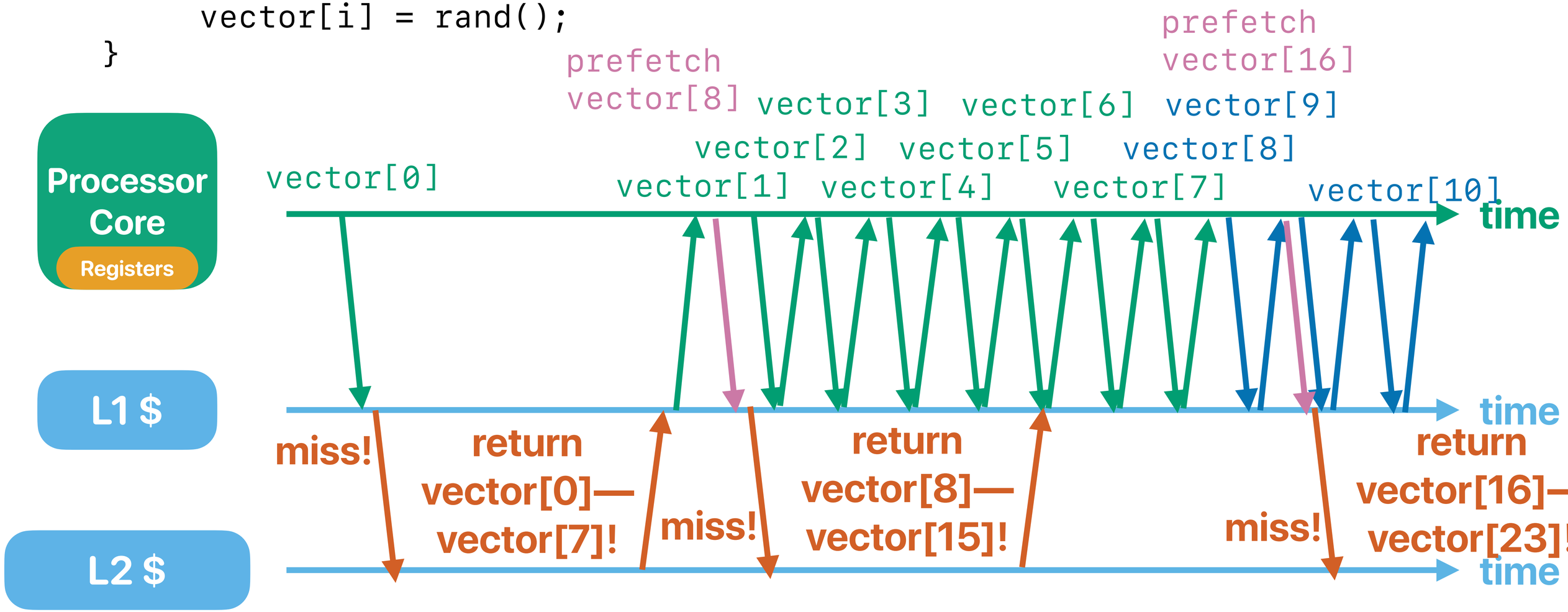
Spatial locality revisited

```
for(i = 0;i < size; i++) {
     vector[i] = rand();
}
```



What if we “pre-“fetch the next line?

```
for(i = 0;i < size; i++) {
     vector[i] = rand();
}
```



Hardware Prefetching

- The hardware identify the access pattern and proactively fetch data/instruction before the application asks for the data/instruction
- Trigger the cache miss earlier to eliminate the miss when the application needs the data/instruction
- The processor can keep track the distance between misses. If there is a pattern between misses, fetch `miss_data_address + offset` for a miss

https://www.pollev.com/hungweitseng close in 1:30

Where can prefetch work effectively?

- How many of the following code snippet can “prefetching” effectively help improving performance?

(1)
```
while(node){
    node = node->next;
}
```

(2)
```
while(++i<100000)
    a[i]=rand();
```

(3)
```
while (root != NULL){
        if (key > root->data)
            root = root->right;

        else if (key < root->data)
            root = root->left;
        else
            return true;
}
```

(4)
```
    for (i = 0; i < 65536; i++) {
      mix_i = ((i * 167) + 13) & 65536;
      results[mix_i]++;
    }
```

A. 0
B. 1
C. 2
D. 3
E. 4



Where can prefetch work effectively?

- How many of the following code snippet can “prefetching” effectively help improving performance?

(1)
```
while(node){
    node = node->next;
}
```
— where the next pointing to is hard to predict

(2)
```
while(++i<100000)
    a[i]=rand();
```

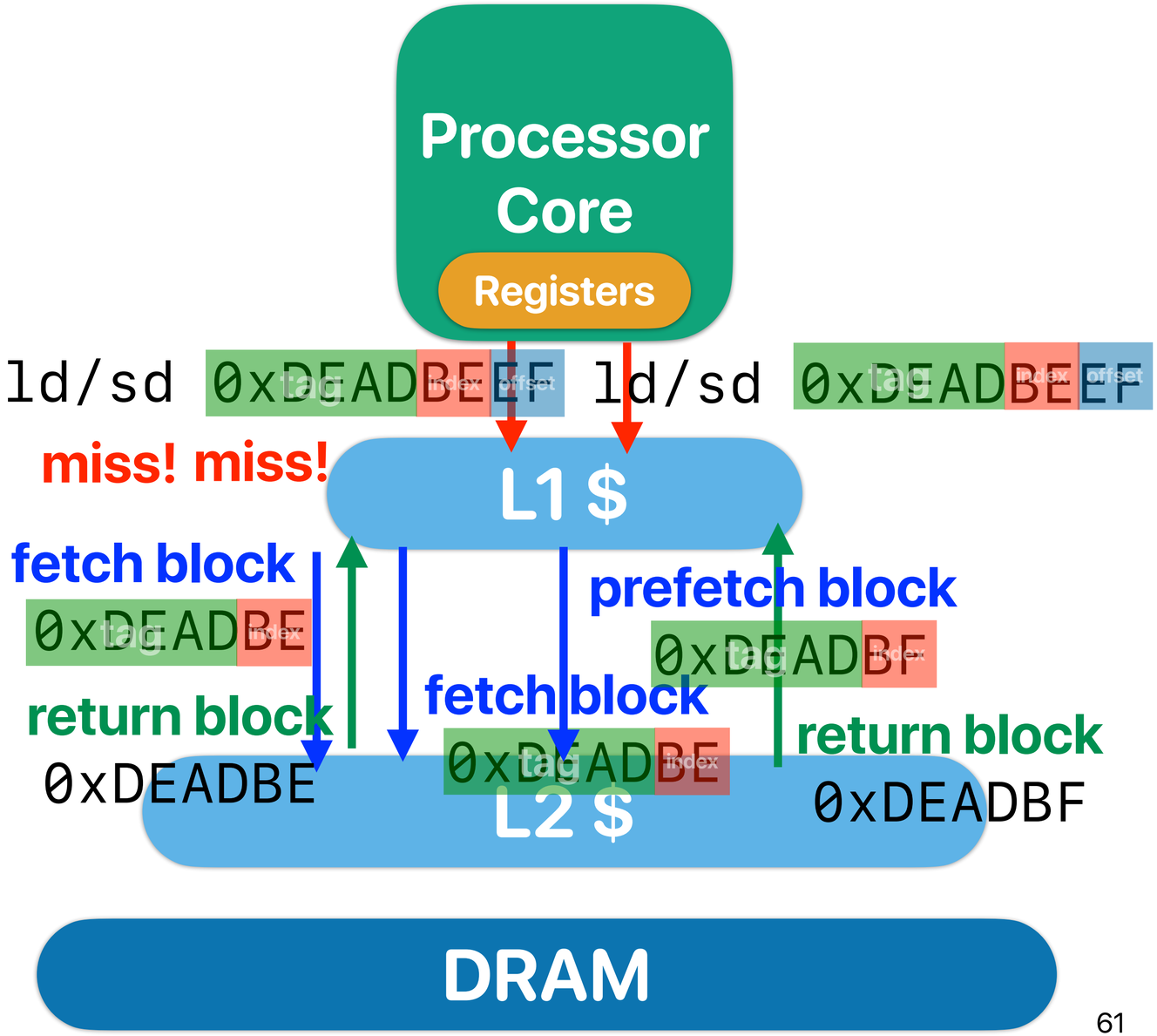
(3)
```
while (root != NULL){
        if (key > root->data)
            root = root->right;

        else if (key < root->data)
            root = root->left;
        else
            return true;
}
```
— where the next node is also hard to predict

(4)
```
    for (i = 0; i < 65536; i++) {
      mix_i = ((i * 167) + 13) & 65536;
      results[mix_i]++;
    }
```
— the stride to the next element is hard to predict…

A. 0
B. 1
C. 2
D. 3
E. 4

What's after prefetching?



NVIDIA Tegra X1 with prefetch

- Size 32KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[8192], b[8192], c[8192], d[8192], e[8192];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
32KB = 4 * 64 * S
S = 128
offset = lg(64) = 6 bits
index = lg(128) = 7 bits
tag = the rest bits

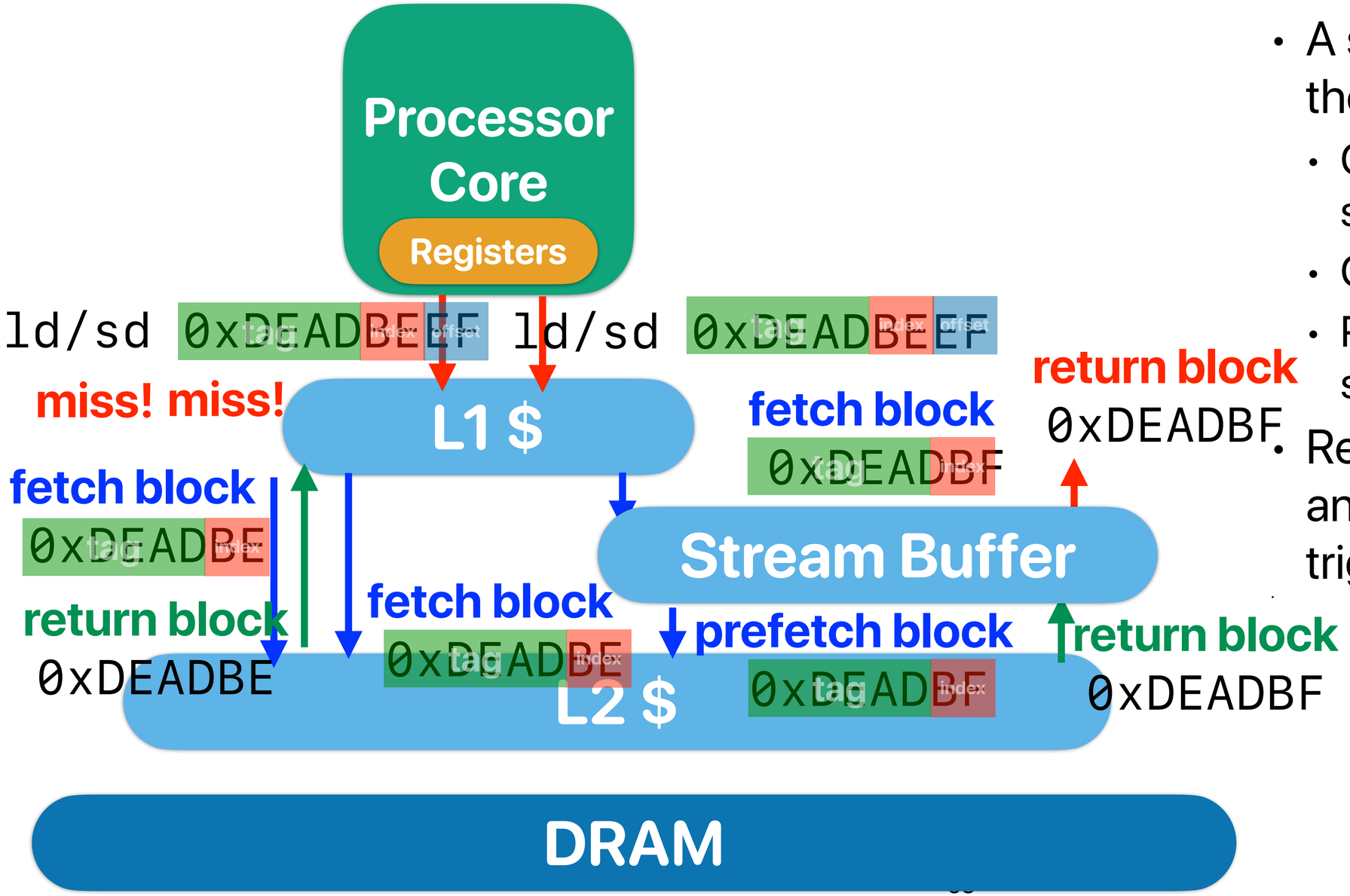
tag index offset

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?	Prefetch
a[0]	0x10000	0b0001000000000000000000	0x8	0x0	Miss		a[8–15]
b[0]	0x20000	0b0010000000000000000000	0x10	0x0	Miss		b[8–15]
c[0]	0x30000	0b0011000000000000000000	0x18	0x0	Miss		c[8–15]
d[0]	0x40000	0b0100000000000000000000	0x20	0x0	Miss		d[8–15]
e[0]	0x50000	0b0101000000000000000000	0x28	0x0	Miss	a[0–7]	e[8–15]
a[1]	0x10008	0b0001000000000000001000	0x8	0x0	Miss	b[0–7]	
b[1]	0x20008	0b0010000000000000001000	0x10	0x0	Miss	c[0–7]	
c[1]	0x30008	0b0011000000000000001000	0x18	0x0	Miss	d[0–7]	
d[1]	0x40008	0b0100000000000000001000	0x20	0x0	Miss	e[0–7]	
e[1]	0x50008	0b0101000000000000001000	0x28	0x0	Miss	a[0–7]	
⋮	⋮	⋮	⋮	⋮	⋮	⋮	

e[8-15] will kick out a[8-15]

100% miss rate!

Stream buffer

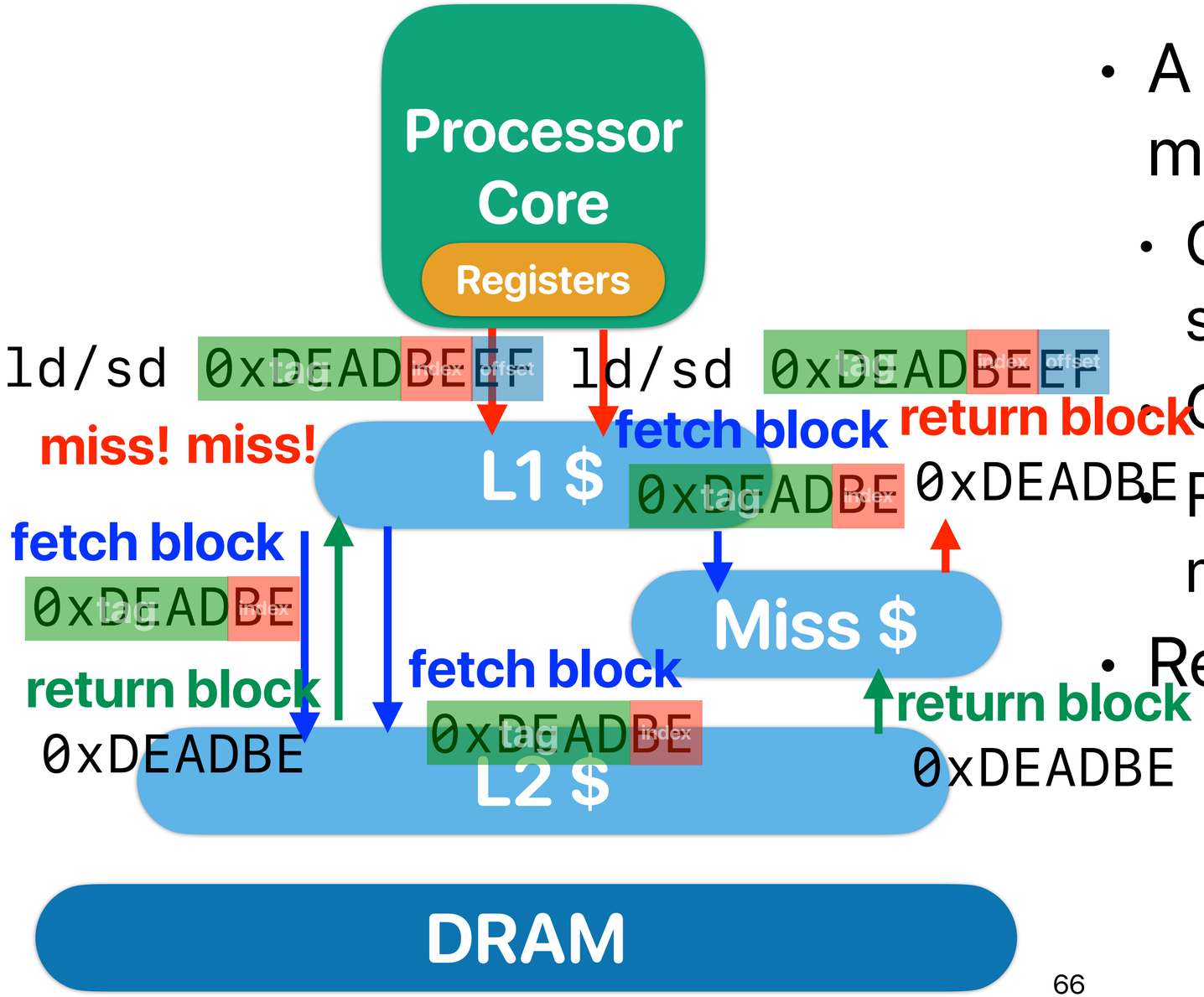


- A small cache that captures the prefetched blocks
  - Can be built as fully associative since it's small
  - Consult when there is a miss
  - Retrieve the block if found in the stream buffer
- Reduce compulsory misses and avoid conflict misses triggered by prefetching

Software Prefetching — through prefetching instructions

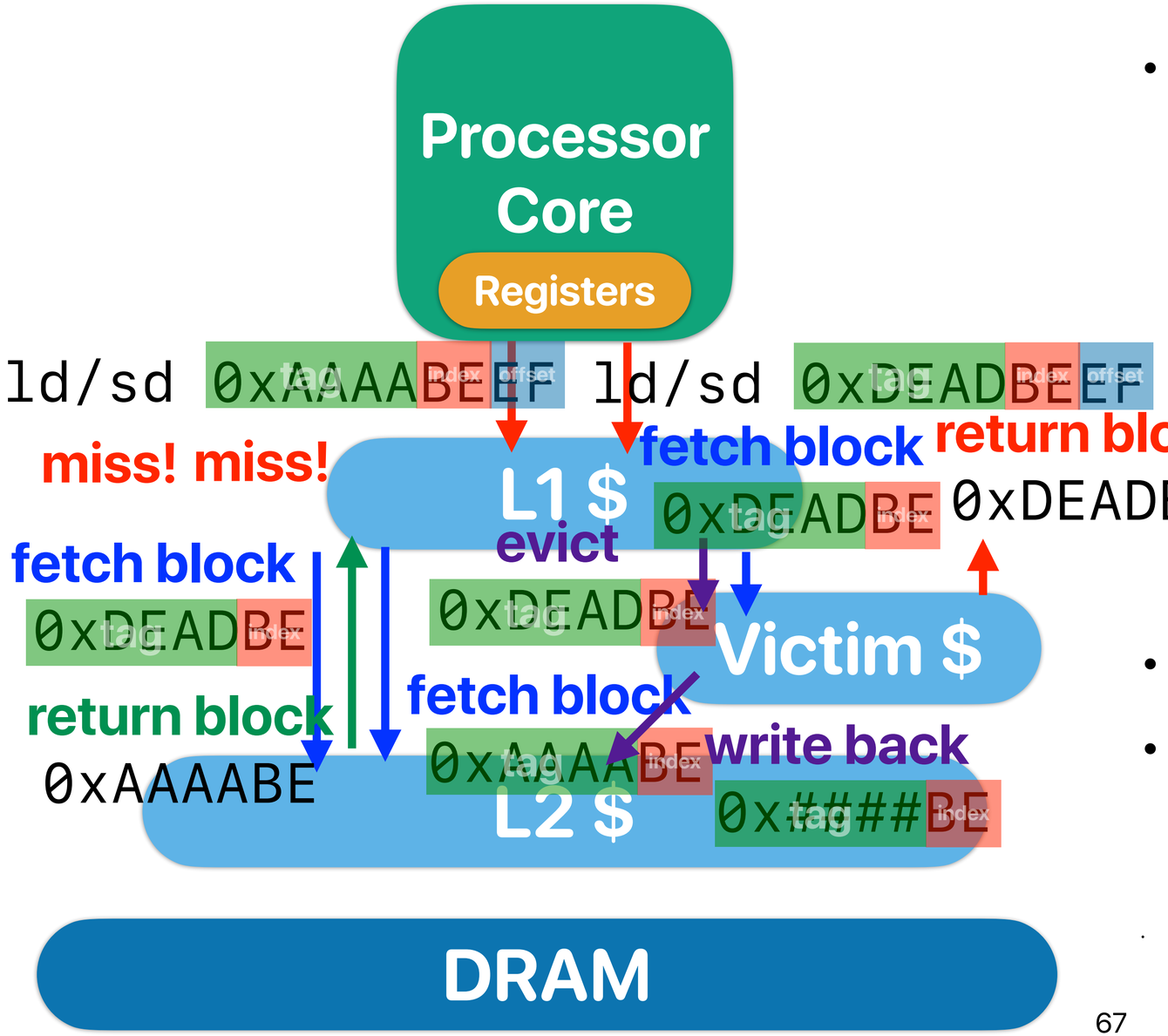
- x86 provide prefetch instructions
- As a programmer, you may insert `_mm_prefetch` in x86 programs to perform software prefetch for your code
- gcc also has a flag "-fprefetch-loop-arrays" to automatically insert software prefetch instructions

Miss cache



- A small cache that captures the missing blocks
  - Can be built as fully associative since it's small
  - Consult when there is a miss
  - Retrieve the block if found in the missing cache
- Reduce conflict misses

Victim cache



- A small cache that captures the evicted blocks
  - Can be built as fully associative since it's small
  - Consult when there is a miss
  - Swap the entry if hit in victim cache
  - Athlon/Phenom has an 8-entry victim cache
- Reduce conflict misses
- Jouppi [1990]: 4-entry victim cache removed 20% to 95% of conflicts for a 4 KB direct mapped data cache

Victim cache v.s. miss caching

- Both of them improves conflict misses
- Victim cache can use cache block more efficiently — swaps when miss
  - Miss caching maintains a copy of the missing data — the cache block can both in L1 and miss cache
  - Victim cache only maintains a cache block when the block is kicked out
- Victim cache captures conflict miss better
  - Miss caching captures every missing block

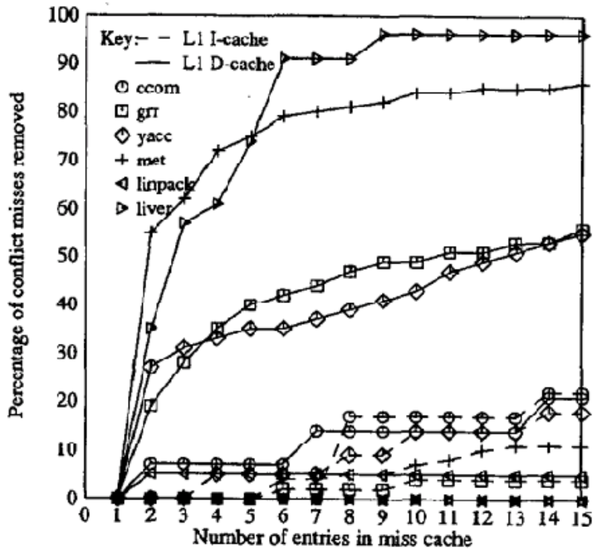


Figure 3-3: Conflict misses removed by miss caching

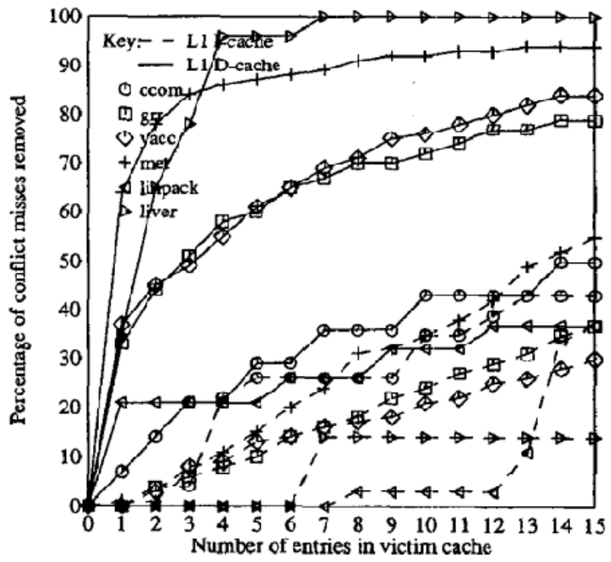


Figure 3-5: Conflict misses removed by victim caching

Which of the following schemes can help Tegra?

- How many of the following schemes mentioned in "improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers" would help NVIDIA's Tegra for the code in the previous slide?
  - ① Missing cache — help improving conflict misses
  - ② Victim cache — help improving conflict misses
  - ③ Prefetch — improving compulsory misses , but can potentially hurt, if we did not do it right
  - ④ Stream buffer — only help improving compulsory misses
  - A. 0
  - B. 1
  - C. 2
  - D. 3
  - E. 4

Takeaways: Optimizing cache performance through hardware

- There is no optimal cache configurations — trade-offs are everywhere
  - Increasing C — (+): capacity misses; (-): cost, access time, power
  - Increasing A — (+): conflict misses; (-): access time, power
  - Increasing B — (+): compulsory misses; (-): miss penalty
- Adding a small buffer alongside the L1 cache can —
  - Virtually add an associative set to frequently used data structures
  - Prefetched blocks won’t cause conflict misses

Takeaways: Optimizing cache performance through hardware

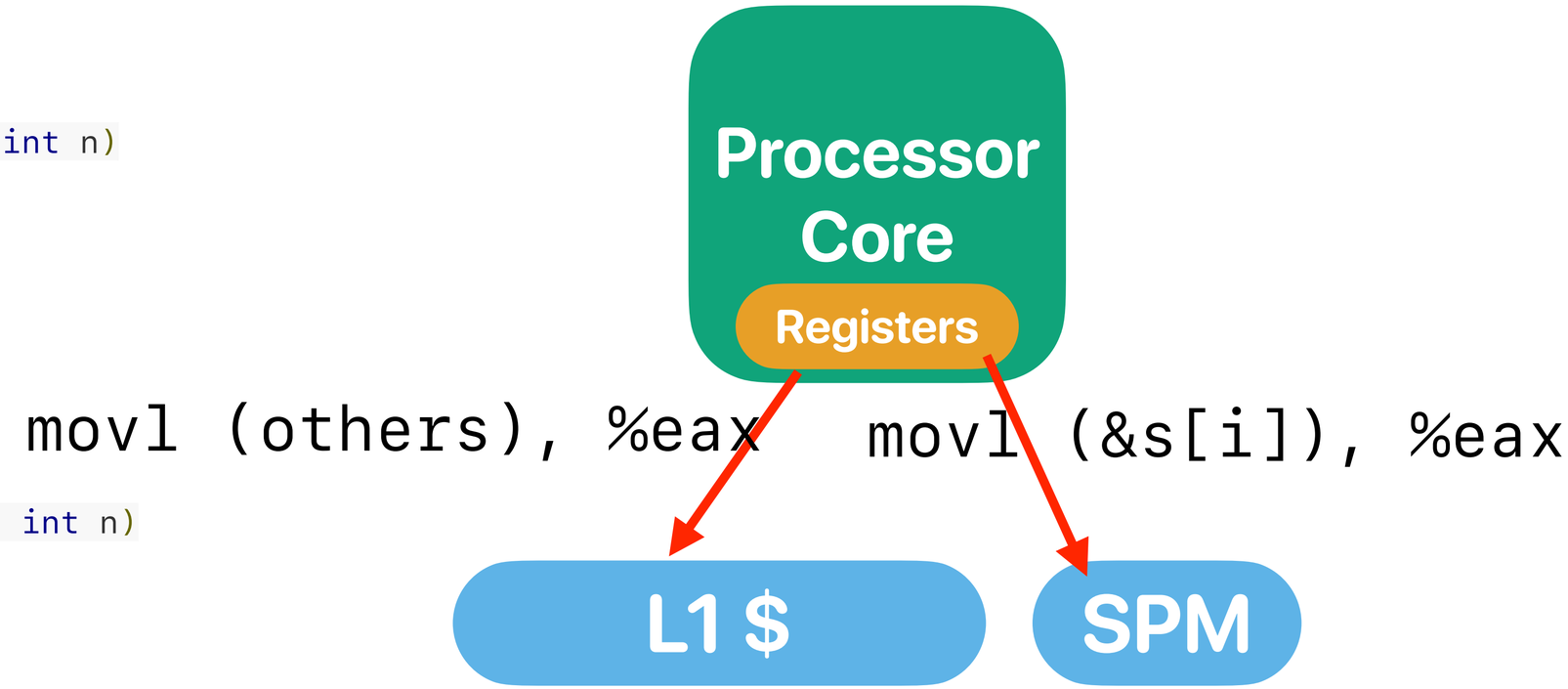
- There is no optimal cache configurations — trade-offs are everywhere
  - Increasing C — (+): capacity misses; (-): cost, access time, **power**
  - Increasing A — (+): conflict misses; (-): access time, **power**
  - Increasing B — (+): compulsory misses; (-): miss penalty
- Adding a small buffer alongside the L1 cache can —
  - Virtually add an associative set to frequently used data structures
  - Prefetched blocks won't cause conflict misses

We need additional search time and power

Implementation of SPM — GPU's shared memory

```
__global__ void staticReverse(int *d, int n)
{
  __shared__ int s[64];
  int t = threadIdx.x;
  int tr = n-t-1;
  s[t] = d[t];
  __syncthreads();
  d[t] = s[tr];
}

__global__ void dynamicReverse(int *d, int n)
{
  extern __shared__ int s[];
  int t = threadIdx.x;
  int tr = n-t-1;
  s[t] = d[t];
  __syncthreads();
  d[t] = s[tr];
}
```



https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/

Takeaways: Optimizing cache performance through hardware

- There is no optimal cache configurations — trade-offs are everywhere
  - Increasing C — (+): capacity misses; (-): cost, access time, power
  - Increasing A — (+): conflict misses; (-): access time, power
  - Increasing B — (+): compulsory misses; (-): miss penalty
- Adding a small buffer alongside the L1 cache can —
  - Virtually add an associative set to frequently used data structures
  - Prefetched blocks won’t cause conflict misses
- Adding a tag-less, programmable small buffer alongside the L1 cache can reduce power consumption

Announcement

- Reading quiz #5 due next **Tuesday** before the lecture
- Assignment #2 released and due **this Thursday**
- Midterm statistics — we still have another midterm and you can drop the lower one

Mean score	67%
Standard	11%
Median score	63%
Maximum	95%

Computer
Science &
Engineering

203



つづく